

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание №5

«Межсервисное взаимодействие посредством очередей сообщений (RabbitMQ)»

Выполнил:
Якшин Артемий
Группа БР1.1

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Подключить и настроить брокер сообщений (RabbitMQ/Kafka) и реализовать на его основе межсервисное взаимодействие в микросервисном приложении «Restaurant Booking» (ЛР2/ЛР3). Подготовить отчёт.

Конкретные требования:

- развернуть брокер сообщений и настроить его в составе проекта (docker-compose);
- организовать асинхронный обмен доменными событиями между сервисами через очередь;
- продемонстрировать работоспособность сквозного сценария «производитель → брокер → потребитель».

Ход работы

1. Выбор брокера сообщений

В проекте «Restaurant Booking» (декомпозиция из ЛР2 на сервисы *auth*, *catalog*, *reservation*, *review* и API Gateway) сервисы до этого взаимодействовали **только синхронно** — по REST с заголовком *X-Internal-Key*. Это создаёт временную связанность: бизнес-операция ждёт ответа зависимого сервиса.

Для асинхронного обмена выбран **RabbitMQ** (протокол AMQP 0-9-1). Обоснование:

- модель `topic-exchange → queue → consumer` напрямую выражает нужный паттерн «публикация события — гибкая маршрутизация по ключу — независимый потребитель»;
- разворачивается одним контейнером с готовым healthcheck, без ZooKeeper/KRaft;
- встроенный Management UI (порт 15672) даёт наглядную проверку exchange, очередей и привязок;
- хорошо ложится на стек проекта Node.js + TypeScript через библиотеку `amqpplib`.

Kafka для текущего масштаба избыточна: не требуются партиционирование, ретеншн лога и перемотка потока. Выбор RabbitMQ согласуется с техническим дизайном из ДЗ4, где брокер и набор событий (*reservation.**, *review.**) уже были спроектированы.

2. Изменения в архитектуре и топология обмена

Добавлено **асинхронное событийное взаимодействие** поверх существующих синхронных контрактов (изменение аддитивное, прежние REST-вызовы не сломаны). Реализован сценарий **уведомлений**: введён новый сервис **notification-service**, который слушает доменные события и формирует уведомления пользователю. Бизнес-операции публикуют события и сразу отвечают клиенту, а обработка идёт независимо у потребителя.

```
Издатели:
reservation-service → reservation.created, reservation.cancelled
review-service      → review.created
                    |
                    ▼
topic exchange "restaurant.events" (durable)
                    |
                    routing key = имя события
```

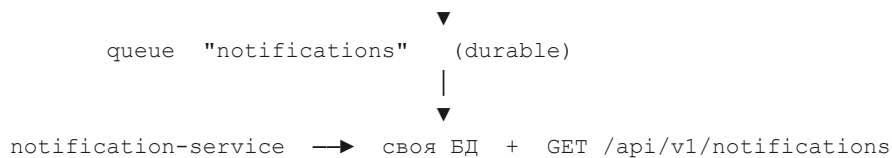


Рис. 1. Асинхронный обмен через RabbitMQ

Параметр	Значение
Exchange	restaurant.events, тип topic, durable: true
Очередь	notifications, durable: true, prefetch = 10, ручной ack
Routing keys	reservation.created, reservation.cancelled, review.created
Привязки очереди	те же три ключа (bindQueue)
Конверт сообщения	{ event, occurred_at, data }, флаг persistent: true

3. Производители событий

В сервисы-издатели добавлен общий модуль шины `src/bus.ts` (одинаковый в *reservation-service* и *review-service*):

- `initBus()` — подключение к брокеру с повторами на старте (RabbitMQ может подниматься дольше сервиса) и объявление topic-exchange;
- `publishEvent(routingKey, data)` — публикация события в конверте {event, occurred_at, data} с флагом persistent;
- публикация — **best effort (fire-and-forget)**: при недоступности брокера ошибка логируется и не ломает HTTP-ответ клиенту (та же «мягкая деградация», что и у синхронных вызовов).

Сервис	Операция	Событие
reservation-service	POST /reservations	reservation.created
reservation-service	DELETE /reservations/:id	reservation.cancelled
review-service	POST /reviews	review.created

4. Потребитель событий — notification-service

Добавлен новый микросервис (порт 8085, своя БД `notifications.sqlite` на volume — принцип *database-per-service* сохранён):

- `src/bus.ts (startConsumer)` объявляет exchange и долговечную очередь, привязывает её к трём routing keys и читает сообщения с ручным ack. При ошибке обработчика — `nack(requeue=false)` (без зацикливания «ядовитого» сообщения); при обрыве — **автоматическое переподключение**;
- `src/index.ts` преобразует событие в человекочитаемое уведомление, сохраняет его в БД и отдаёт пользователю через `GET /api/v1/notifications` (защита клиентским JWT). В API Gateway добавлен соответствующий маршрут.

Событие	Заголовок уведомления
reservation.created	«Бронирование подтверждено»

reservation.cancelled	«Бронирование отменено»
review.created	«Спасибо за отзыв»

5. Конфигурация и запуск

В `docker-compose.yml` добавлены: контейнер брокера `rabbitmq:3.13-management` с `healthcheck` (`rabbitmq-diagnostics ping`) и портами `5672` (AMQP) / `15672` (UI); сервис `notification-service` с `healthcheck`; переменная `RABBITMQ_URL` для издателей и потребителя; зависимости `depends_on: rabbitmq (service_healthy)`; том `notification-data`. Итог — 7 контейнеров (5 сервисов + `gateway` + `rabbitmq`).

```
cd "BP1.1/Якшин Артемий/labs/lab2"
docker compose up --build
# API:          http://localhost:3000/api/v1
# RabbitMQ UI:  http://localhost:15672 (guest/guest)
```

6. Проверка работоспособности

Выполнен сквозной сценарий через `gateway`: регистрация пользователя → создание отзыва и брони → отмена брони → чтение уведомлений. Все три события прошли цепочку *производитель → exchange → очередь → потребитель → БД → API*.

Логи издателей — события опубликованы в `exchange`:

```
reservation-service | [bus] connected to RabbitMQ, exchange "restaurant.events"
reservation-service | [bus] -> reservation.created {"reservation_id":4,...}
reservation-service | [bus] -> reservation.cancelled {"reservation_id":4,...}
review-service      | [bus] -> review.created {"review_id":5,"rating":5}
```

Логи потребителя — события приняты из очереди и обработаны:

```
notification-service | [bus] consuming "notifications" <- "restaurant.events"
                      keys=[reservation.created, reservation.cancelled, review.created]
notification-service | [notification] stored "Спасибо за отзыв" for user 4
notification-service | [notification] stored "Бронирование подтверждено" for user 4
notification-service | [notification] stored "Бронирование отменено" for user 4
```

Топология в брокере (`rabbitmqctl`):

```
$ rabbitmqctl list_exchanges name type
restaurant.events  topic

$ rabbitmqctl list_bindings source_name routing_key destination_name
restaurant.events  reservation.created  notifications
restaurant.events  reservation.cancelled notifications
restaurant.events  review.created       notifications
```

Ответ `GET /api/v1/notifications` — уведомления, собранные из очереди:

```
{
  "data": [
    { "type": "reservation.cancelled", "title": "Бронирование отменено",
      "message": "Бронь #4 на 2026-12-31 19:00 отменена." },
    { "type": "reservation.created", "title": "Бронирование подтверждено",
      "message": "Столик #1 забронирован на 2026-12-31 19:00 (гостей: 2)."},
    { "type": "review.created", "title": "Спасибо за отзыв",
      "message": "Ваш отзыв на ресторан #2 (оценка 5/5) опубликован." }
  ],
  "total": 3, "page": 1, "limit": 20
}
```

7. Надёжность доставки

- **Долговечность:** `exchange` и очередь — `durable`, сообщения — `persistent`; переживают перезапуск брокера.
- **Подтверждение:** ручной `ack` только после успешной обработки; при ошибке — `nack(requeue=false)`.
- **Переподключение:** потребитель восстанавливает соединение в цикле; издатели подключаются с повторами на старте.
- **Деградация:** публикация события не блокирует и не ломает основную операцию (`best effort`).
- **Развитие:** `dead-letter exchange` для упавших сообщений, идемпотентность потребителя (дедупликация по `event_id`) и `outbox`-паттерн у издателей.

Вывод

В микросервисное приложение «Restaurant Booking» добавлено асинхронное межсервисное взаимодействие на базе RabbitMQ. Развёрнут и настроен брокер сообщений в составе `docker-compose`; сервисы *reservation* и *review* публикуют доменные события в `topic-exchange restaurant.events`, а новый *notification-service* потребляет их из долговечной очереди и формирует уведомления пользователю. Работоспособность подтверждена сквозным сценарием: события прошли весь путь от издателя через брокер до потребителя, что проверено логами сервисов, состоянием брокера (`rabbitmqctl`) и ответом API. Тем самым синхронные REST-вызовы дополнены событийной, слабосвязанной моделью обмена.